TEMPORADA XIV

CI NICARAGUA · 2026

DOCUMENTO DE LECTURA

07

// Unidad II · Desarrollo Backend con Java

APIS, ENDPOINTS Y VERBOS HTTP

CÓMO CONVERSAN LAS APPS

EN ESTA LECTURA

01 ¿Qué es una API?
02 Petición y respuesta HTTP
03 Endpoints y URLs
04 Los verbos HTTP
05 Códigos de estado
06 Cómo recibe datos un endpoint
07 Un controlador REST completo
08 Probar la API con Postman



ELABORADO POR

MARVIN RAFAEL MEZA PINEDA

Facilitador · Centro Nacional de Innovación y Tecnología
Francisco “El Chele” Moreno

Fundamentos de Desarrollo
Backend con Java
Octubre 2026

CAPÍTULO 01

1 ¿QUÉ ES UNA API?

> **API** significa *Application Programming Interface* (Interfaz de Programación de Aplicaciones). Es el **contrato** que define cómo otras aplicaciones pueden comunicarse con nuestro backend.

Cuando construimos un backend con Spring Boot, la API establece tres cosas: **qué peticiones** puede hacer un cliente (una app móvil, una página web hecha en React, otro sistema), **qué datos** debe enviar y **qué respuesta** recibirá. Si el cliente respeta el contrato, no necesita saber nada de cómo está programado el backend por dentro.

EL RESTAURANTE, OTRA VEZ

En la Lectura 06 usamos un restaurante para explicar las capas por dentro. Ahora míralo **desde la mesa del cliente**: el cliente no entra a la cocina. Lee el **menú**, que dice qué se puede pedir y cómo, y hace su pedido al **mesero**, que regresa con el plato. El menú y el mesero juntos son la **API**; cada plato del menú es un **endpoint**; la cocina (service, repository y base de datos) queda oculta.

APIs REST

Hay varias formas de diseñar una API. La más usada en el desarrollo web es **REST** (*Representational State Transfer*), y es la que construiremos con Spring Boot. Una API REST sigue estas ideas:

Principio	¿Qué significa?
Todo es un recurso	Los datos se organizan como recursos con nombre: estudiantes, cursos, notas. Cada recurso tiene su propia dirección (URL).
Los verbos dicen la acción	La URL indica *sobre qué* se trabaja y el método HTTP indica *qué hacer*: leer, crear, actualizar o eliminar.
Sin estado (*stateless*)	Cada petición trae toda la información necesaria. El servidor no "recuerda" la petición anterior del mismo cliente.
Representación en JSON	Los recursos viajan como texto JSON, un formato ligero que entiende cualquier lenguaje de programación.

CAPÍTULO 02

2 PETICIÓN Y RESPUESTA HTTP

> **HTTP** es el protocolo con el que viajan los mensajes en la web. Toda comunicación es un par: el cliente envía una **petición** (*request*) y el servidor devuelve una **respuesta** (*response*).

Aunque las herramientas nos lo ocultan, un mensaje HTTP es simplemente **texto** con una estructura fija. Así se ve una petición para crear un estudiante y la respuesta del servidor:

PETICIÓN · CLIENTE → SERVIDOR

```
POST /api/v1/estudiantes HTTP/1.1    ← línea inicial
Host: localhost:8080                 ← encabezados
Content-Type: application/json
Accept: application/json
                                     ← línea vacía
{                                    ← cuerpo
  "nombre": "Ana López",
  "edad": 17
}
```

RESPUESTA · SERVIDOR → CLIENTE

```
HTTP/1.1 201 Created                 ← línea de estado
Content-Type: application/json       ← encabezados
Location: /api/v1/estudiantes/1

{                                    ← cuerpo
  "id": 1,
  "nombre": "Ana López",
  "edad": 17
}
```

Parte	En la petición	En la respuesta
Línea inicial	Verbo + ruta + versión de HTTP.	Versión + **código de estado** + su nombre.
Encabezados	Datos sobre la petición: qué formato se envía (`Content-Type`), qué formato se espera (`Accept`), credenciales.	Datos sobre la respuesta: formato del cuerpo, dónde quedó el recurso creado (`Location`).
Cuerpo (*body*)	Los datos que se envían. Lo llevan `POST`, `PUT` y `PATCH`; `GET` y `DELETE` normalmente no.	Los datos que se devuelven, casi siempre en JSON. Puede ir vacío.

EL ENCABEZADO MÁS IMPORTANTE

`Content-Type: application/json` le avisa al servidor que el cuerpo es JSON. Si falta, Spring Boot no sabrá cómo leerlo y responderá con un error `415 Unsupported Media Type`. Postman lo agrega solo cuando eliges el formato JSON.

CAPÍTULO 03

3 ENDPOINTS Y URLS

> Un **endpoint** es un punto de entrada a la API: la combinación de un **verbo HTTP** y una **ruta**. Si la API es el menú completo, cada endpoint es un plato.

Anatomía de una URL

```
http://  localhost  :8080  /api  /v1  /estudiantes/10  ?activo=true
```

- **Protocolo**: http o https (seguro)
- **Host**: el servidor (en producción, un dominio)
- **Puerto**: 8080 es el de Spring Boot
- **Prefijo**: separa la API del resto del sitio
- **Versión**: permite cambiar la API sin romper clientes
- **Recurso e id**: el estudiante número 10
- **Parámetros de consulta**: filtros opcionales

Un endpoint es verbo + ruta

La misma ruta puede ser **varios endpoints distintos**, según el verbo:

Endpoint	Qué hace
`GET /api/v1/estudiantes`	Lista todos los estudiantes.
`POST /api/v1/estudiantes`	Crea un estudiante nuevo.
`GET /api/v1/estudiantes/10`	Obtiene al estudiante 10.
`PUT /api/v1/estudiantes/10`	Actualiza al estudiante 10.
`DELETE /api/v1/estudiantes/10`	Elimina al estudiante 10.

Buenas prácticas para nombrar endpoints

× EVITA

- Verbos en la ruta: `/obtenerEstudiantes`, `/borrarEstudiante/5`
- Singular: `/estudiante`
- Mayúsculas y espacios: `/Estudiantes_Activos`
- Ids en parámetros: `/estudiantes?id=5`

✓ PREFIERE

- Sustantivos; el verbo HTTP ya dice la acción: `GET /estudiantes`
- Plural, porque es una colección: `/estudiantes`
- Minúsculas y guiones: `/estudiantes-activos`
- El id en la ruta: `/estudiantes/5`

Cuando un recurso pertenece a otro, la ruta refleja esa jerarquía. Por ejemplo, `GET /api/v1/cursos/3/estudiantes` se lee "los estudiantes del curso 3".

CAPÍTULO 04

4 LOS VERBOS HTTP

> Los verbos (o métodos) HTTP le dicen al servidor **qué quiere hacer** el cliente con el recurso. Los principales corresponden a las operaciones **CRUD**: *Create*, *Read*, *Update*, *Delete*.

Verbo	CRUD	En Spring Boot	Propósito	Cuerpo	Seguro	Idempotente
`GET`	Leer	`@GetMapping`	Consulta uno o varios recursos. Nunca modifica datos.	No	Sí	Sí
`POST`	Crear	`@PostMapping`	Envía datos para crear un recurso nuevo.	Sí	No	No
`PUT`	Actualizar	`@PutMapping`	Reemplaza **completo** un recurso existente.	Sí	No	Sí
`PATCH`	Actualizar	`@PatchMapping`	Modifica **solo algunos** campos del recurso.	Sí	No	No*
`DELETE`	Eliminar	`@DeleteMapping`	Elimina un recurso según su id.	No	No	Sí

* `PATCH` puede ser idempotente o no, según cómo se implemente.

Seguro e idempotente

- **Seguro**: el verbo no cambia nada en el servidor; solo lee. Solo `GET` lo es. Por eso un `GET` se puede repetir, guardar en caché o abrir en el navegador sin riesgo.

- **Idempotente**: repetir la misma petición varias veces deja el servidor **igual que hacerla una vez**. Si borras al estudiante 5 dos veces, sigue borrado una sola vez. En cambio, enviar dos veces el mismo `POST` crea **dos** estudiantes.

PUT frente a PATCH

PUT · REEMPLAZA TODO

```
PUT /api/v1/estudiantes/10
{
  "nombre": "Ana López",
  "edad": 18
}
Se envían TODOS los campos. Si falta
uno, se entiende que queda vacío.
```

PATCH · CAMBIA SOLO LO ENVIADO

```
PATCH /api/v1/estudiantes/10
{
  "edad": 18
}

Solo cambia la edad; el nombre
se queda como estaba.
```

CAPÍTULO 05

5 CÓDIGOS DE ESTADO

> Cada respuesta trae un **código de estado** de tres cifras que resume qué pasó con la petición. El cliente lo revisa antes de leer el cuerpo.

El primer dígito indica la **familia**, y con eso ya sabes de quién es la responsabilidad:

Familia	Tipo	Cómo leerla
`2xx`	Éxito	Todo salió bien.
`3xx`	Redirección	El recurso está en otra dirección.
`4xx`	Error del cliente	La petición está mal: datos inválidos, ruta inexistente, falta de permisos. **El cliente debe corregirla.**
`5xx`	Error del servidor	La petición era válida, pero el backend falló. **Hay que corregir el código del servidor.**

Los códigos que más usarás

Código	Nombre	¿Cuándo devolverlo?
`200`	OK	Petición exitosa con datos en la respuesta: un `GET` o un `PUT`.
`201`	Created	Se creó un recurso nuevo, después de un `POST`.
`204`	No Content	Éxito, pero sin nada que devolver; típico de un `DELETE`.
`400`	Bad Request	El cliente envió datos incorrectos: JSON mal formado, un campo obligatorio vacío, una edad negativa.

`404`	Not Found	La ruta no existe, o el recurso pedido no existe (no hay estudiante con id 99).
`405`	Method Not Allowed	La ruta existe, pero no acepta ese verbo (un `DELETE` donde solo hay `GET`).
`409`	Conflict	La petición choca con el estado actual: por ejemplo, un correo que ya está registrado.
`500`	Internal Server Error	Una excepción no controlada en nuestro código.

UN ERROR COMÚN

Devolver siempre `200` con un mensaje de texto como "Error: no se encontró el estudiante". Para quien consume la API eso es un **éxito**, y su programa seguirá como si todo estuviera bien. El código de estado debe decir la verdad: si no existe, es `404`.

CAPÍTULO 06

6 CÓMO RECIBE DATOS UN ENDPOINT

> Un cliente puede enviar información en tres lugares de la petición, y Spring Boot tiene una anotación para leer cada uno.

Anotación	Lee datos de…	Ejemplo de petición	Úsala para
`@PathVariable`	La ruta	`GET /estudiantes/10`	Identificar un recurso.
`@RequestParam`	Los parámetros de consulta	`GET /estudiantes?edadMinima=18`	Filtros y opciones.
`@RequestBody`	El cuerpo (JSON)	`POST /estudiantes` + JSON	Enviar los datos de un recurso.

```java
// @PathVariable: el {id} de la ruta se copia en el parámetro id
@GetMapping("/{id}")
public Estudiante obtenerPorId(@PathVariable Long id) { ... }

// @RequestParam: ?edadMinima=18  ->  edadMinima vale 18
// required = false: el filtro es opcional; si no viene, vale null
@GetMapping
public List<Estudiante> listar(@RequestParam(required = false) Integer edadMinima) { ... }

// @RequestBody: el JSON del cuerpo se convierte en un objeto Estudiante
@PostMapping
public Estudiante crear(@RequestBody Estudiante nuevo) { ... }
```

¿POR QUÉ INTEGER Y NO INT?

Un `int` no puede quedar "vacío": siempre tiene un número. `Integer` es su versión como objeto y sí puede valer `null`, que es justo lo que ocurre cuando el cliente no envía el filtro opcional.

CAPÍTULO 07

7 UN CONTROLADOR REST COMPLETO

> Juntamos todo en un controlador con los cinco endpoints del recurso estudiantes, devolviendo el código de estado correcto en cada caso.

Para concentrarnos en HTTP, guardaremos los estudiantes en una `List` en memoria. En un proyecto real, esa lista se reemplaza por las capas **service** y **repository** de la Lectura 06, y los endpoints no cambian. Primero, la clase que representa al recurso:

model/Estudiante.java

```java
public class Estudiante {

    private Long id;
    private String nombre;
    private int edad;

    public Estudiante() { }   // Spring lo necesita para convertir el JSON en objeto

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public int getEdad() { return edad; }
    public void setEdad(int edad) { this.edad = edad; }
}
```

ResponseEntity: controlar la respuesta

Si un método devuelve directamente un objeto, Spring siempre responde `200`. Para elegir el código de estado, el método devuelve un `ResponseEntity`, que es la respuesta completa: código, encabezados y cuerpo.

Expresión	Respuesta que genera
`ResponseEntity.ok(estudiante)`	`200 OK` con el estudiante en el cuerpo.
`ResponseEntity.status(HttpStatus.CREATED).body(e)`	`201 Created` con el recurso creado.
`ResponseEntity.noContent().build()`	`204 No Content`, sin cuerpo.
`ResponseEntity.notFound().build()`	`404 Not Found`, sin cuerpo.
`ResponseEntity.badRequest().build()`	`400 Bad Request`, sin cuerpo.

controller/EstudianteController.java

```java
@RestController
@RequestMapping("/api/v1/estudiantes")       // ruta base del recurso
public class EstudianteController {

    private final List<Estudiante> estudiantes = new ArrayList<>();  // "base de datos"
    private long siguienteId = 1;

    // 1. GET /api/v1/estudiantes  ·  GET /api/v1/estudiantes?edadMinima=18
    @GetMapping
    public List<Estudiante> listar(@RequestParam(required = false) Integer edadMinima) {
        if (edadMinima == null) {
            return estudiantes;                          // 200 OK con todos
        }
        List<Estudiante> filtrados = new ArrayList<>();
        for (Estudiante e : estudiantes) {
            if (e.getEdad() >= edadMinima) {
                filtrados.add(e);
            }
        }
        return filtrados;                                // 200 OK con los filtrados
    }

    // 2. GET /api/v1/estudiantes/10
    @GetMapping("/{id}")
    public ResponseEntity<Estudiante> obtenerPorId(@PathVariable Long id) {
        Estudiante e = buscar(id);
        if (e == null) {
            return ResponseEntity.notFound().build();     // 404
        }
        return ResponseEntity.ok(e);                      // 200
    }

    // 3. POST /api/v1/estudiantes
    @PostMapping
    public ResponseEntity<Estudiante> crear(@RequestBody Estudiante nuevo) {
        if (nuevo.getNombre() == null || nuevo.getNombre().isBlank() || nuevo.getEdad() < 14)
{
            return ResponseEntity.badRequest().build();   // 400: datos inválidos
        }
        nuevo.setId(siguienteId++);
        estudiantes.add(nuevo);
        return ResponseEntity.status(HttpStatus.CREATED).body(nuevo);   // 201
    }

    // 4. PUT /api/v1/estudiantes/10
    @PutMapping("/{id}")
    public ResponseEntity<Estudiante> actualizar(@PathVariable Long id,
                                                 @RequestBody Estudiante datos) {
        Estudiante e = buscar(id);
        if (e == null) {
            return ResponseEntity.notFound().build();     // 404
        }
        e.setNombre(datos.getNombre());
        e.setEdad(datos.getEdad());
        return ResponseEntity.ok(e);                      // 200
    }

    // 5. DELETE /api/v1/estudiantes/10
```

```java
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> eliminar(@PathVariable Long id) {
        Estudiante e = buscar(id);
        if (e == null) {
            return ResponseEntity.notFound().build();     // 404
        }
        estudiantes.remove(e);
        return ResponseEntity.noContent().build();        // 204
    }

    // Método auxiliar privado: busca por id con un for-each
    private Estudiante buscar(Long id) {
        for (Estudiante e : estudiantes) {
            if (e.getId().equals(id)) {
                return e;
            }
        }
        return null;
    }
}
```

Lo que conviene notar

- **`@RequestMapping` en la clase** define la ruta base una sola vez; cada método solo agrega lo que cambia (`/{id}`).
- **`ResponseEntity<Void>`** en el `DELETE` indica que la respuesta no lleva cuerpo.
- **Los ids se comparan con `.equals()`**: `Long` es un objeto, igual que `String`. Comparar objetos con `==` es el mismo error que vimos en la Lectura 02.
- **El método `buscar` es `private`**: no tiene anotación de mapeo, así que no es un endpoint. Es un ayudante interno que evita repetir el mismo ciclo en tres métodos.
- **Las validaciones** (nombre vacío, menos de 14 años) aquí son mínimas y están en el controlador para que el ejemplo sea corto. En un proyecto real van en el **service**, como vimos en la Lectura 06.

El contrato de nuestra API

Así documentaríamos la API para quien la vaya a usar. Esta tabla **es** el contrato del capítulo 1:

Verbo	Ruta	Cuerpo	Respuestas posibles
`GET`	`/api/v1/estudiantes`	—	`200` lista (filtro opcional `?edadMinima=`)
`GET`	`/api/v1/estudiantes/{id}`	—	`200` estudiante · `404` no existe
`POST`	`/api/v1/estudiantes`	Estudiante	`201` creado · `400` datos inválidos
`PUT`	`/api/v1/estudiantes/{id}`	Estudiante	`200` actualizado · `404` no existe
`DELETE`	`/api/v1/estudiantes/{id}`	—	`204` eliminado · `404` no existe

CAPÍTULO 08

8 PROBAR LA API CON POSTMAN

> El navegador solo hace peticiones `GET`. Para probar `POST`, `PUT` y `DELETE` usamos **Postman**, una aplicación gratuita para enviar peticiones HTTP y ver las respuestas.

1. Con la aplicación Spring Boot corriendo, abre Postman y pulsa `+` para crear una petición nueva.
2. A la izquierda de la barra de dirección elige el verbo (**POST**) y escribe la URL: `http://localhost:8080/api/v1/estudiantes`
3. Ve a la pestaña **Body**, marca **raw** y en el menú de la derecha elige **JSON**. Escribe el cuerpo: `{"nombre": "Ana López", "edad": 17}`
4. Pulsa **Send**. Abajo verás el **código de estado** (debe ser `201 Created`), el tiempo de respuesta y el cuerpo con el `id` asignado.
5. Prueba el resto del contrato: `GET` a la misma URL, `GET` a `/1` y a `/99` (debe dar `404`), un `PUT` a `/1` con datos nuevos y un `DELETE` a `/1` (debe dar `204`).

SIN POSTMAN

VS Code tiene extensiones como **Thunder Client** o **REST Client** que hacen lo mismo sin salir del editor. Y Windows incluye `curl` en la terminal: `curl http://localhost:8080/api/v1/estudiantes`

Resumen

Concepto	Recuerda
API	El contrato que dice qué se puede pedir al backend, con qué datos y qué se recibe.
REST	Recursos con nombre, verbos HTTP para la acción, sin estado y datos en JSON.
Endpoint	Verbo + ruta. Rutas con sustantivos en plural; el id va en la ruta.
Verbos	`GET` lee · `POST` crea · `PUT` reemplaza · `PATCH` modifica · `DELETE` elimina.
Códigos	`2xx` éxito · `4xx` error del cliente · `5xx` error del servidor. Deben decir la verdad.
Datos de entrada	`@PathVariable` ruta · `@RequestParam` filtros · `@RequestBody` JSON.
ResponseEntity	Permite elegir el código de estado de cada respuesta.

Referencias

MDN Web Docs. *HTTP request methods* y *HTTP response status codes*. developer.mozilla.org/es/docs/Web/HTTP

Spring. *Building a RESTful Web Service*. spring.io/guides/gs/rest-service

Spring. *Spring Framework Reference: Annotated Controllers*. docs.spring.io/spring-framework/reference/web/webmvc



MARVIN RAFAEL MEZA PINEDA

Facilitador · Fundamentos de Desarrollo Backend con Java

Centro Nacional de Innovación y Tecnología Francisco “El Chele” Moreno · Temporada XIV · 2026